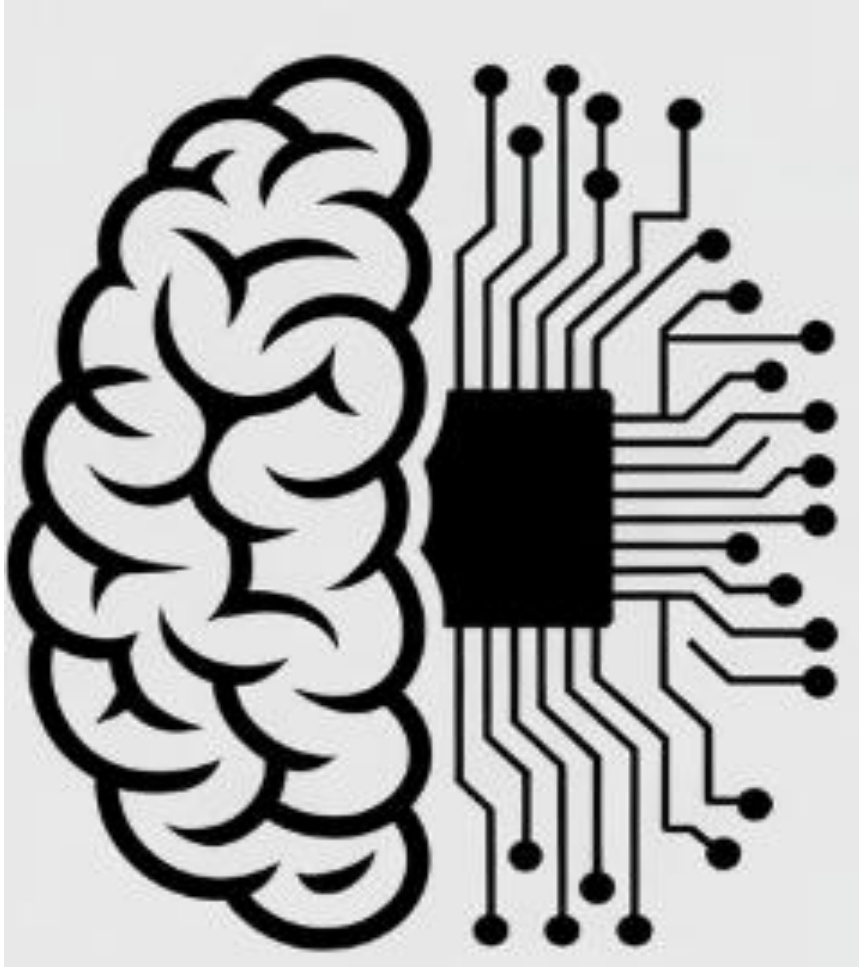




Perceptron Learning

Dr. Sobia Tariq Javed

Supervised Learning - Classification

Training Data			
inputs			
$x_1 = [x_{11} \ x_{12} \ x_{13} \ x_{14}]$	targets / labels / ground truth $y_1 = 1$	predictions $\hat{y}_1 = 1$	
$x_2 = [x_{21} \ x_{22} \ x_{23} \ x_{24}]$	$y_2 = 2$	$\hat{y}_2 = 2$	
$x_3 = [x_{31} \ x_{32} \ x_{33} \ x_{34}]$	$y_3 = 1$	$\hat{y}_3 = 2$	
.			
.			
.			
$x_n = [x_{n1} \ x_{n2} \ x_{n3} \ x_{n4}]$	$y_n = 3$	$\hat{y}_n = 1$	

- We need to find a function that maps x and y for any of them.

$$\hat{y}_i = f(x_i; \theta)$$

Learnable
parameters

- How do we "learn" the parameters of this function?
- We choose ones that makes the following quantity small:

$$\sum_{i=1}^n \mathcal{L}(\hat{y}_i, y_i)$$

Loss function

Supervised Learning

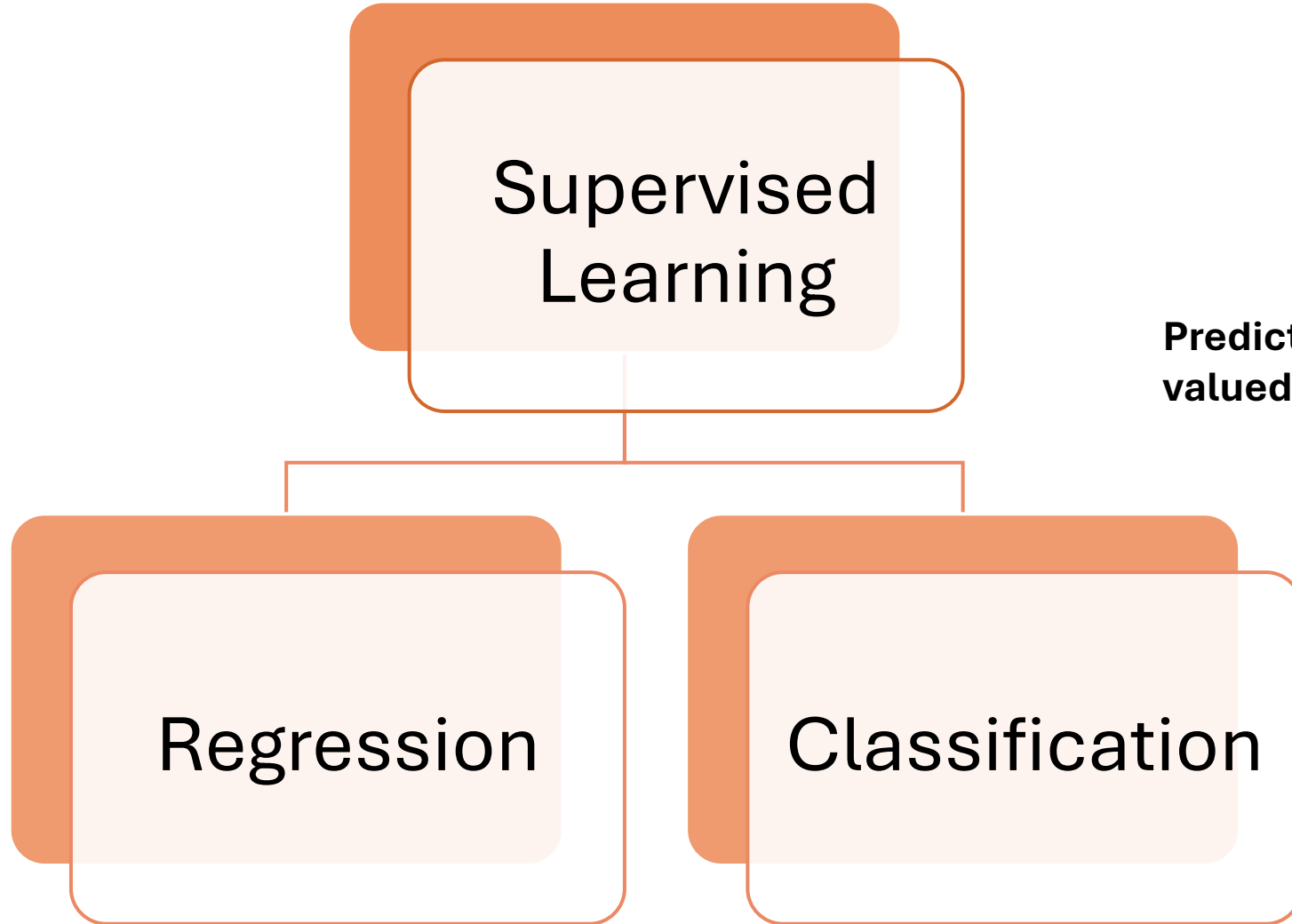
**Predict continuous
valued output (price)**

**Supervised
Learning**

**Predict discrete
valued output (0 or 1)**

Regression

Classification

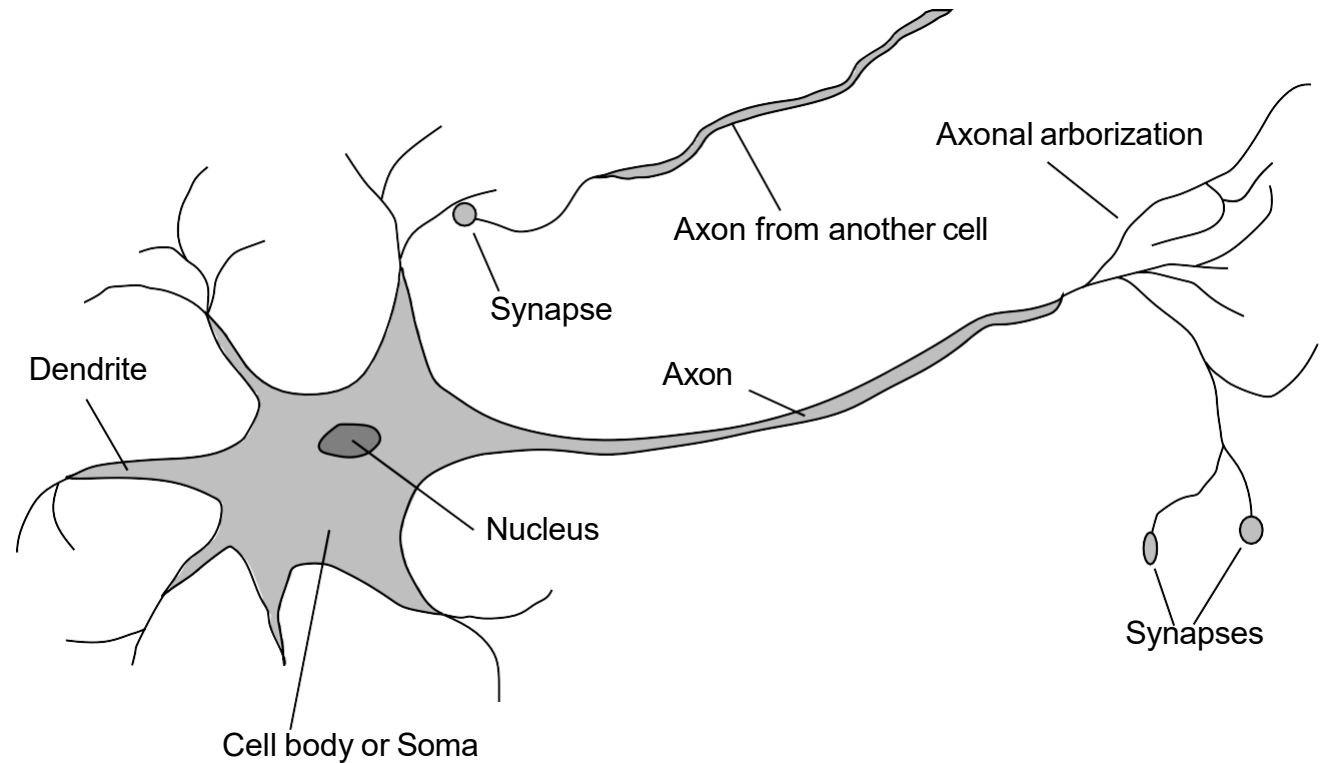


Biological Inspirations

- Humans perform complex tasks like:
 - vision,
 - motor control,
 - or language understanding very well.
- One way to build intelligent machines is to try to imitate the (organizational principles of) human brain.

Biological Neuron

- A **biological neuron** is a **nerve cell in the brain and nervous system** that processes and transmits information. It is the basic unit that allows the brain **to think, learn, and control the body.**
- Humans have **billions of neurons** that communicate with each other through **electrical and chemical signals.**



Main parts of Biological Neuron

Dendrites

- These are **branch-like structures**.
- They **receive signals** from other neurons.

Cell Body (Soma)

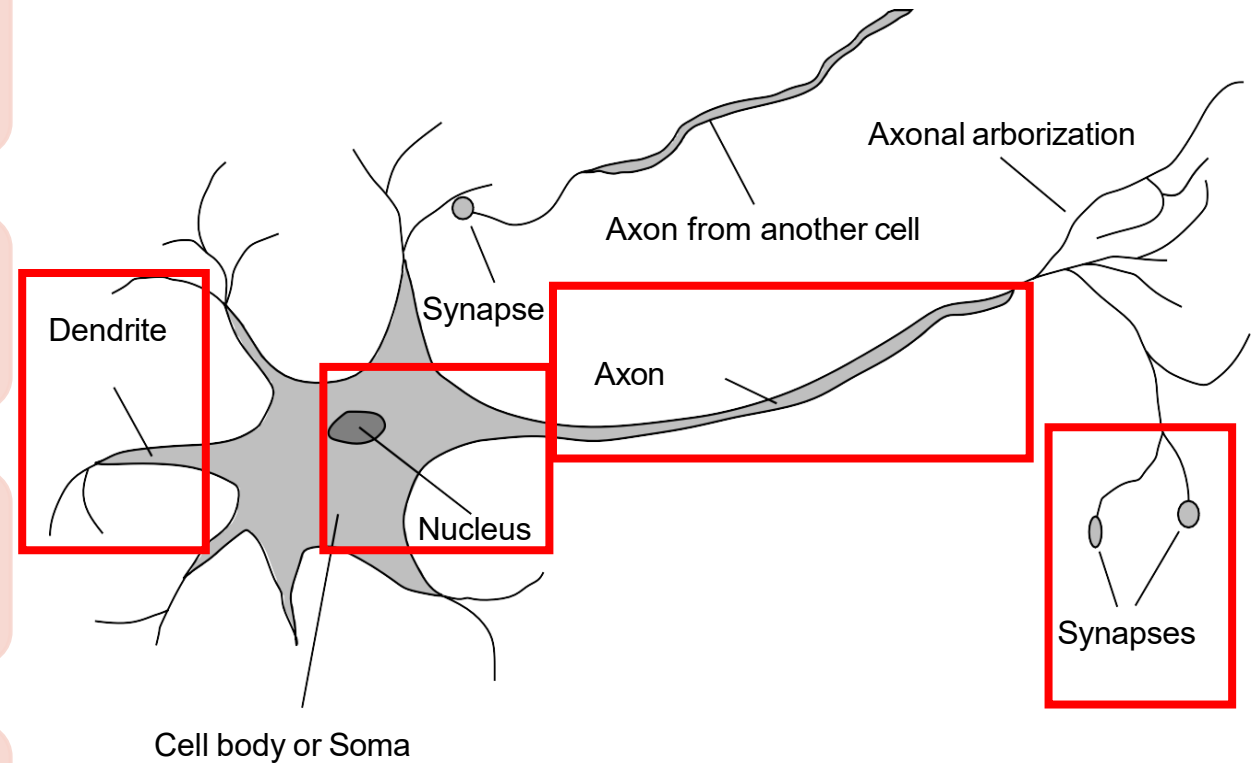
- The **central part** of the neuron.
- It **processes the signals** received from dendrites.

Axon

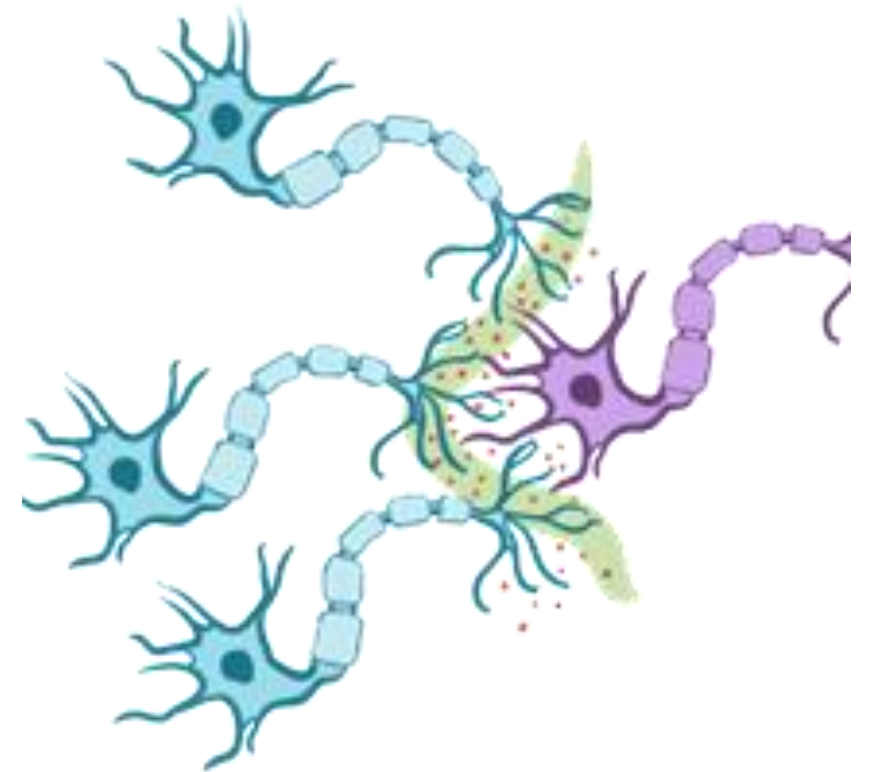
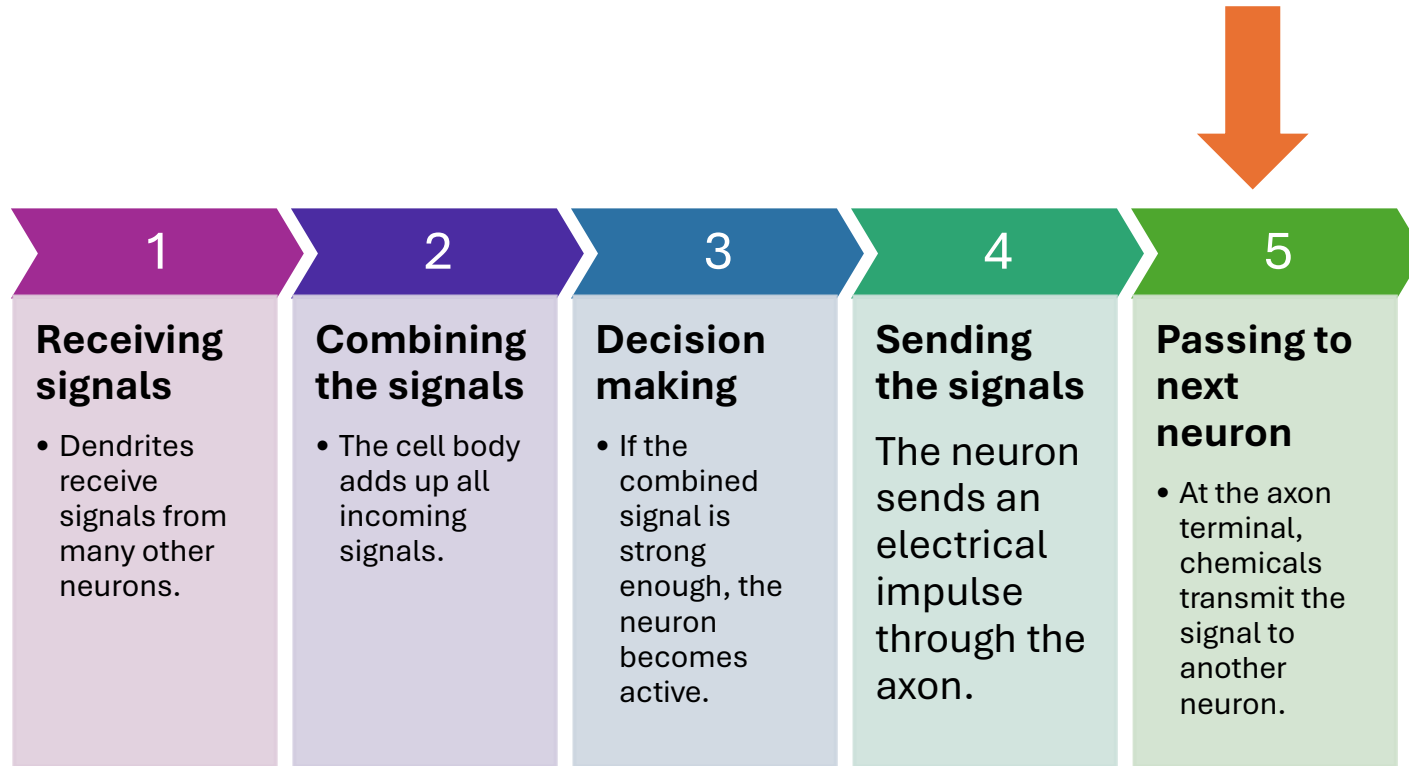
- A **long fiber** that carries signals away from the neuron.

Axon Terminals / Synapse

- The end part of the neuron.
- It **passes the signal to the next neuron** using chemicals called **neurotransmitters**.



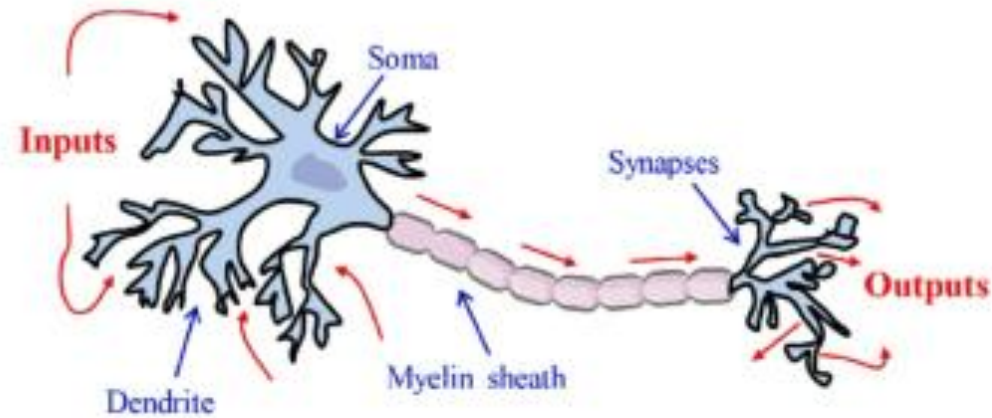
How a Biological Neuron Works



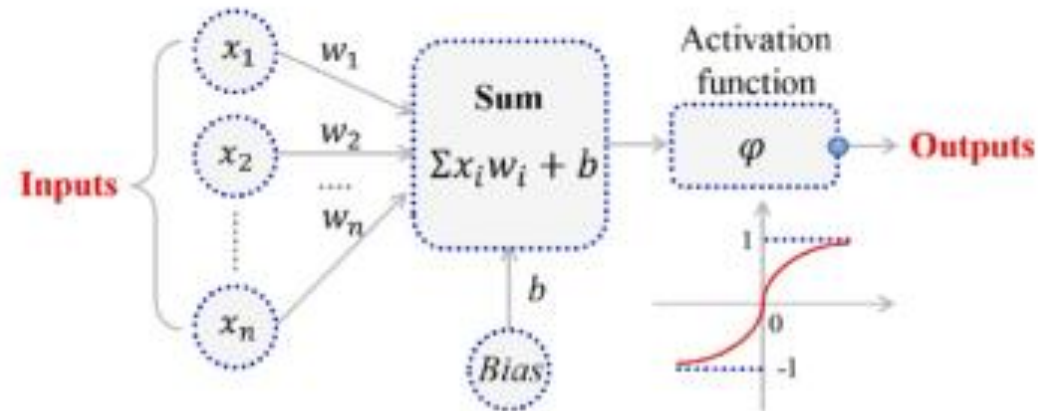
Artificial Neural Network

- **Computational models** inspired by the **human brain**:
- **Massively parallel, distributed system**, made up of simple processing units (neurons)
- **Synaptic connection strengths** among neurons are used to store the acquired knowledge.
- Knowledge is acquired by the network from its environment through **a learning process**

Artificial Neural Network



(a) Biological neuron

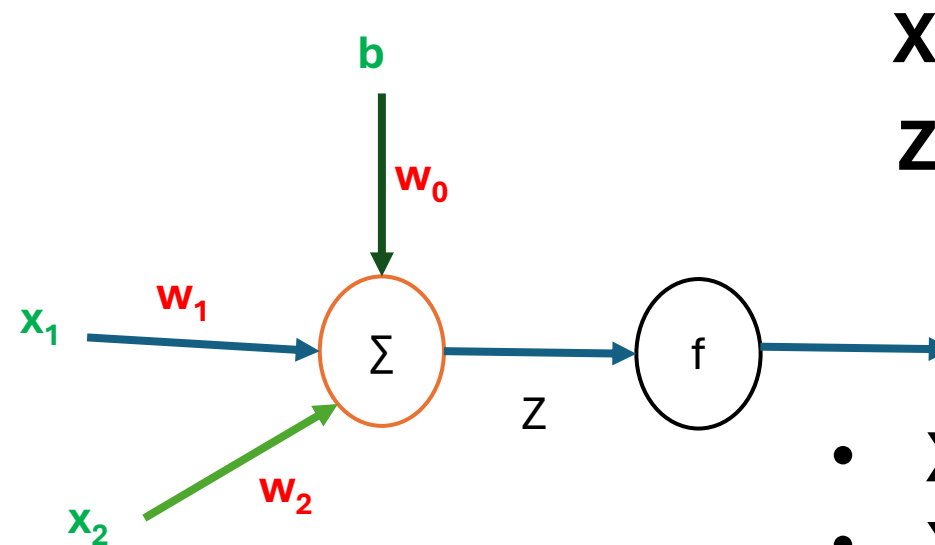


(b) Artificial neuron

Suppose we have 2 inputs X_1 and X_2 with bias b .

Perceptron Learning

- **Perceptron Learning** is a supervised learning algorithm used for **binary classification**.
- It is used to classify data that is linearly separable.

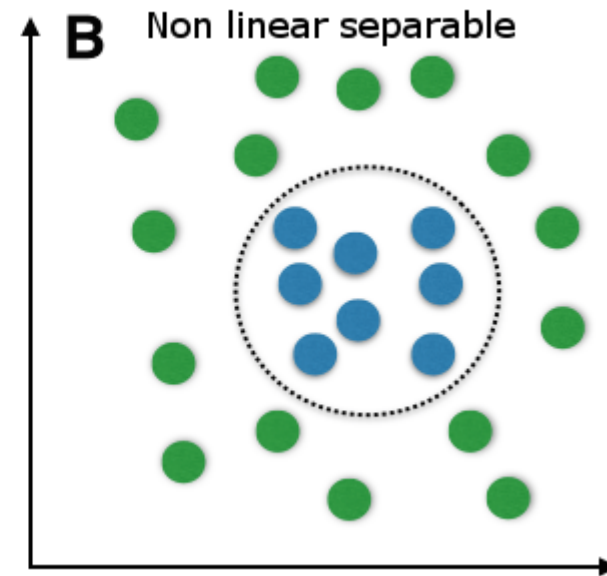
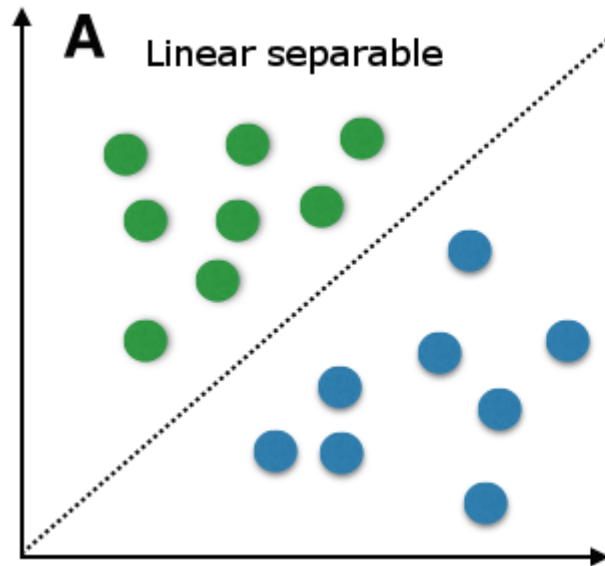


$$X_0 = b$$

$$Z = X_0W_0 + X_1W_1 + X_2W_2$$

- X_0, X_1 and X_2 are inputs
 - $X_0 = 1$ and is called bias
 - W_0, W_1 and W_2 are weights
- **f is the activation function**

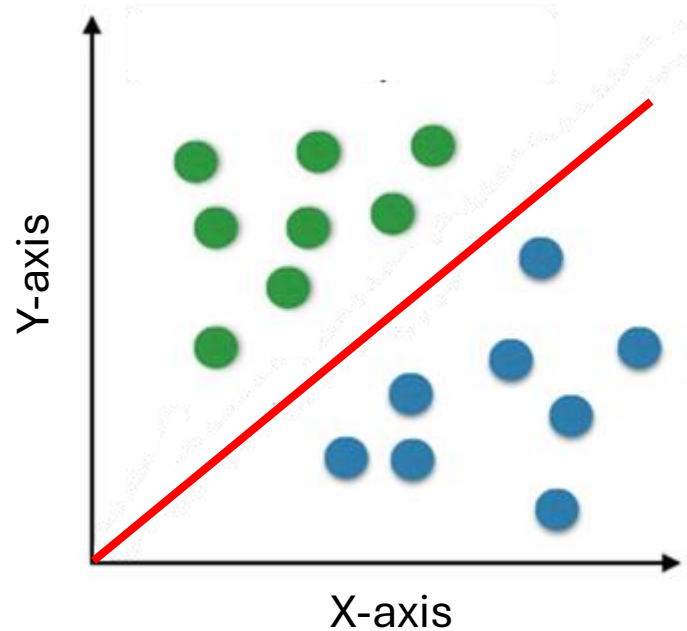
Linear Separable Vs. Non-Linear Separable data



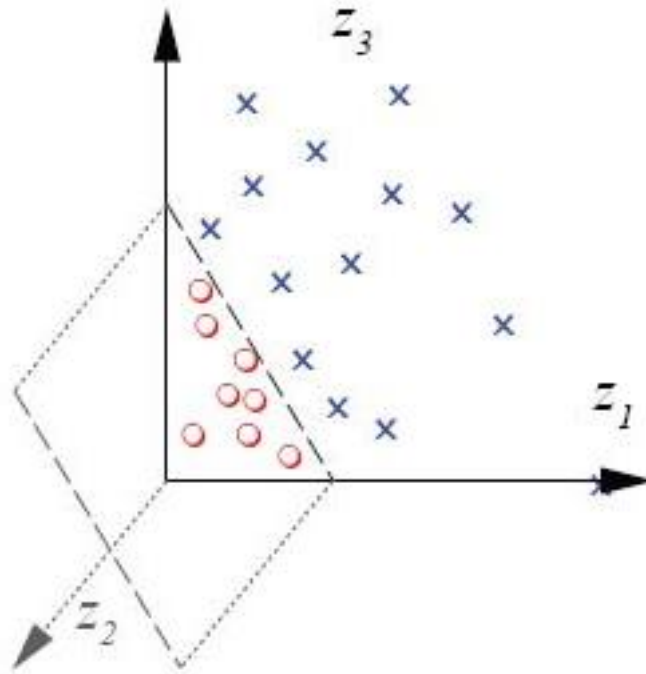
Linear Separable

- $Z = X_0W_0 + X_1W_1 + X_2W_2$ Since $X_0 = 1$
- $Z = W_0 + X_1W_1 + X_2W_2$ eq(1)
- If I replace $X_1 = x$, $X_2 = y$, $W_0 = C$, $W_1 = A$, $W_2 = B$
 - So eq(1) will become
 - $Z = C + Ax + By$ (2 inputs x & y)
- $Z = C + Ax + By + Dz$ (3 inputs x, y & z)
- 2 inputs $\rightarrow$ line
- 3 inputs $\rightarrow$ plane
- 4 inputs $\rightarrow$ hyperplane

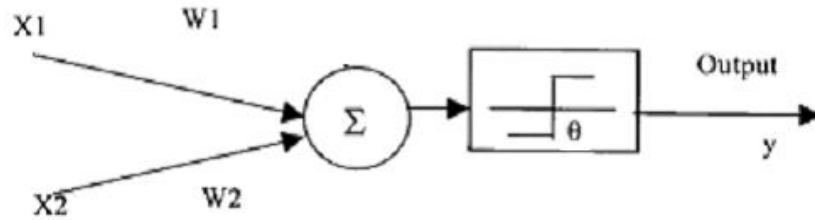
Linear Separable (Line)



Data Separable by hyper plane



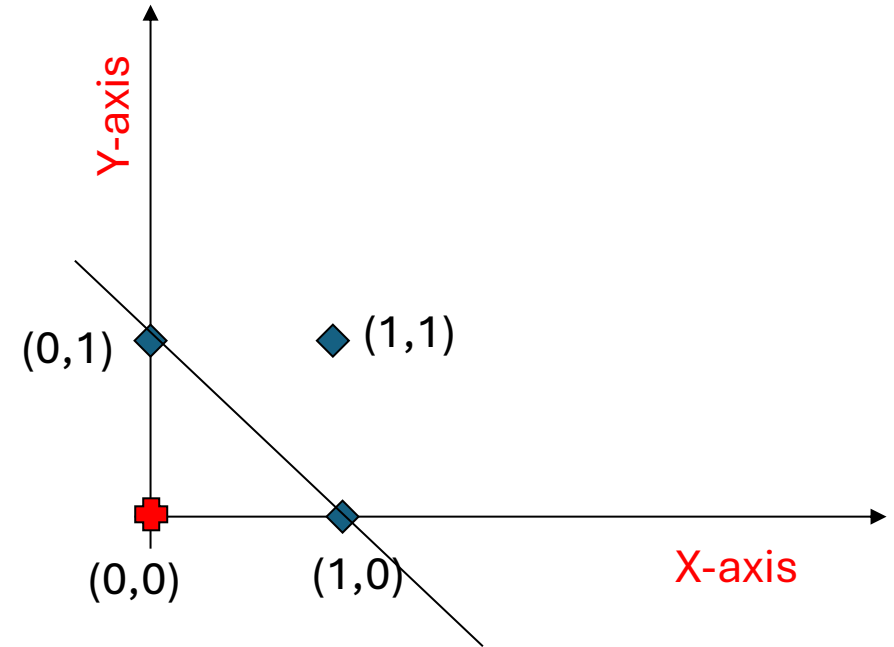
OR Gate



$$Z = X_0W_0 + X_1W_1 + X_2W_2$$
$$X_0=0, W_1=W_2=1$$

- A typical OR gate truth table is shown below

Input		Actual Output	Z
X	Y		
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	2



AND Gate

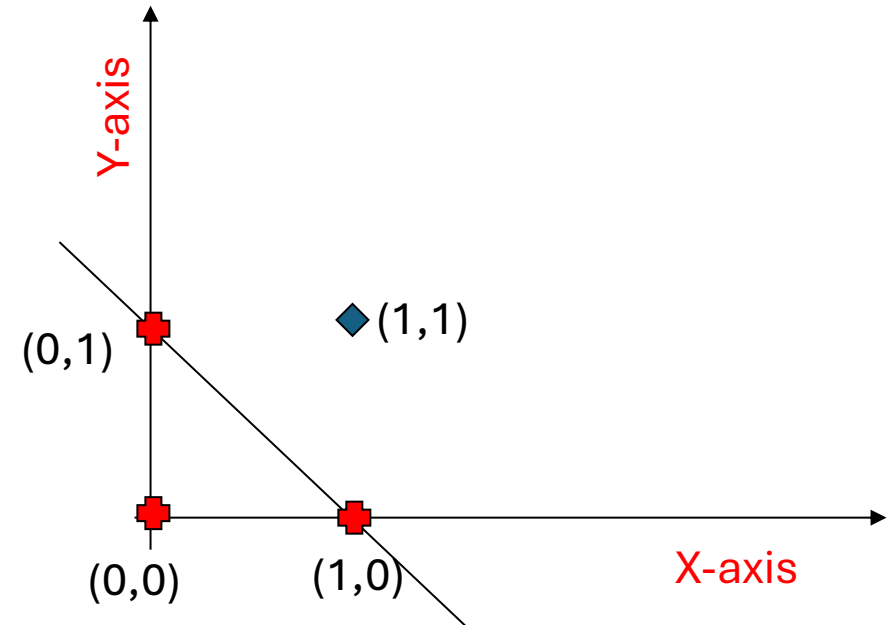
$$Z = X_0W_0 + X_1W_1 + X_2W_2$$

$$W_0=0, W_1=W_2=1$$

Find value of θ

- A typical AND gate truth table is shown below

Input		Actual Output	Z
X	Y		
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	2



HW

Weights for AND Gate

$$Z = X_0W_0 + X_1W_1 + X_2W_2$$

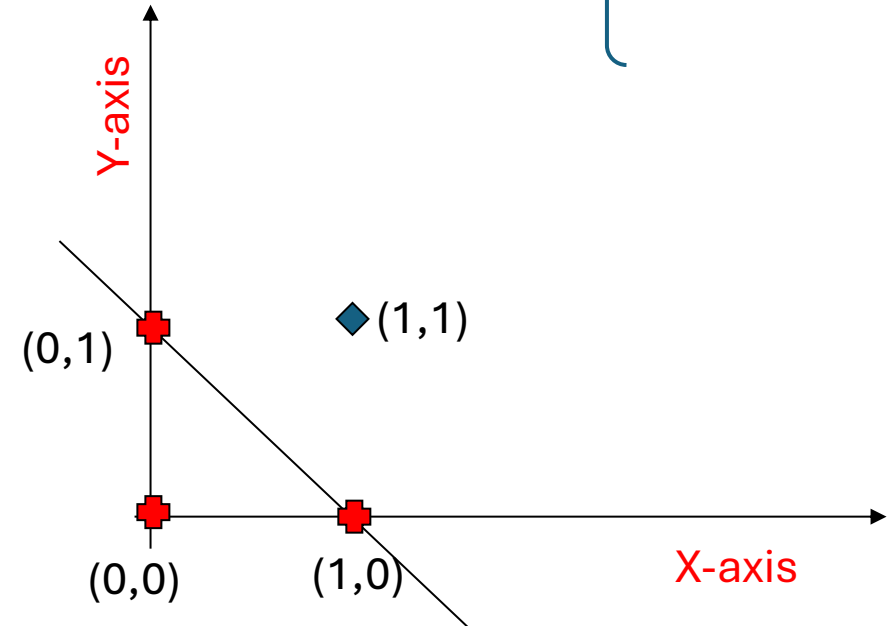
$$X_0=1, \theta=0$$

Find values of W_0 , W_1 and W_2

- A typical AND gate truth table is shown below

Input		Actual Output	Z
X	Y		
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	2

$$F = \begin{cases} 1 & \text{if } Z \geq \theta \\ 0 & \text{if } Z < \theta \end{cases}$$



HW

- Find values of w_1, w_2 and b for NAND function with (threshold) $\theta = 0$

Input		Actual Output	Z
X	Y		
0	0	1	
0	1	1	
1	0	1	
1	1	0	

$$F = \left\{ \begin{array}{l} 1 \text{ if } Z \geq \theta \\ 0 \text{ if } Z < \theta \end{array} \right\}$$

CW

X1	X2	y
0	0	0
0	1	0
1	0	1
1	1	1

- Show the mathematical working of perceptron learning by taking the case in figure. First two columns are the input values for X1 and X2 and the third column is the Actual output.
- Learning rate = 0.2
- Threshold = $\theta = 0.5$, initially take $W_1 = W_2 = 1$, $b=0$
- Predicted output = $\hat{y} = f(Z)$ where f is the step function
- $Z = W_1X_1 + W_2X_2 + W_3b$
- Next weight adjustment = $W_n + \Delta W_n$ (delta rule)
- Change in weight = $\Delta W_n = \text{learning rate} * (\text{Actual output} - \text{predicted output}) * X_n$
- In Figure.
- First two columns are input vectors x1 and x2 and last column is the actual output y.
- Show the complete iterations for acquiring the actual output y?

CW

Learning rate = 0.2, **Threshold** = $\theta = 0.5$, $W_1 = W_2 = 1$, $b = 0$

$Z = W_1 X_1 + W_2 X_2 + W_3 b$

Predicted output = $\hat{y} = f(Z)$, where **f** is the step function

Next weight adjustment = $W_n + \Delta W_n$

$\Delta W_n = \text{learning rate} * (\text{Actual output} - \text{predicted output}) * X_n$

X1	X2	y
0	0	0
0	1	0
1	0	1
1	1	1

Iteration # 1

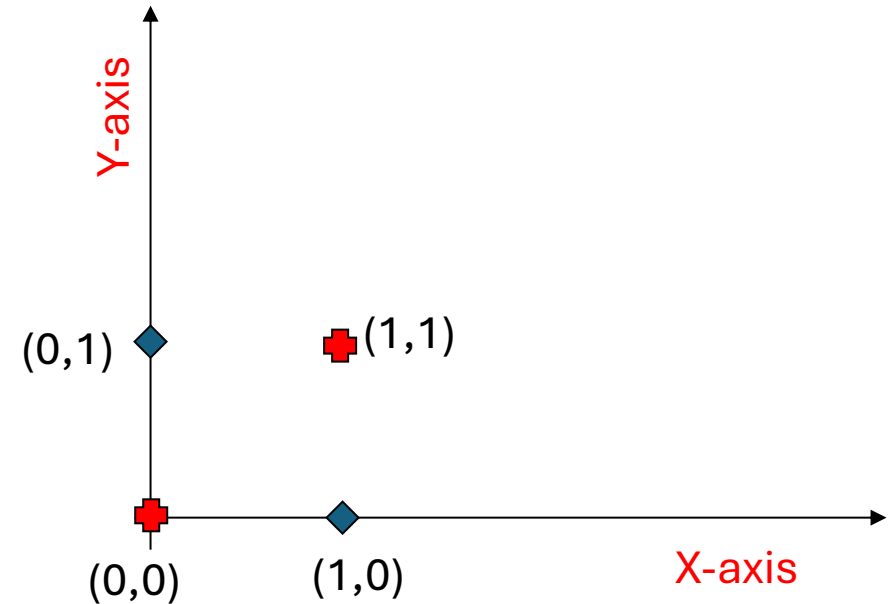
X_1	X_2	W_1	W_2	y	$\hat{y}$	ΔW_1	ΔW_2
0	0	1	1	0			
0	1			0			
1	0			1			
1	1			1			

XOR Gate

$$Z = X_0W_0 + X_1W_1 + X_2W_2$$
$$W_0=0, W_1=W_2=1$$

- A typical XOR gate truth table is shown below

Input		Actual Output	Z
X	Y		
0	0	0	
0	1	1	
1	0	1	
1	1	0	



Activation functions

- An activation function is a mathematical function applied to the weighted sum of inputs in a neuron

$$z = \sum_{i=1}^n w_i x_i + b$$

$a(z)$

- **Role of Activation Function**

- Determines whether a neuron activates and to what extent (output magnitude).
- Introduces nonlinearity into the neural network.

Qualities of Good Activation functions

- Non-Linear
- Differentiable
- Computationally inexpensive
- Zero centered
- Non-Saturating

Common Activation functions

1. Binary Step Function

(a “threshold” or “Heaviside” function)

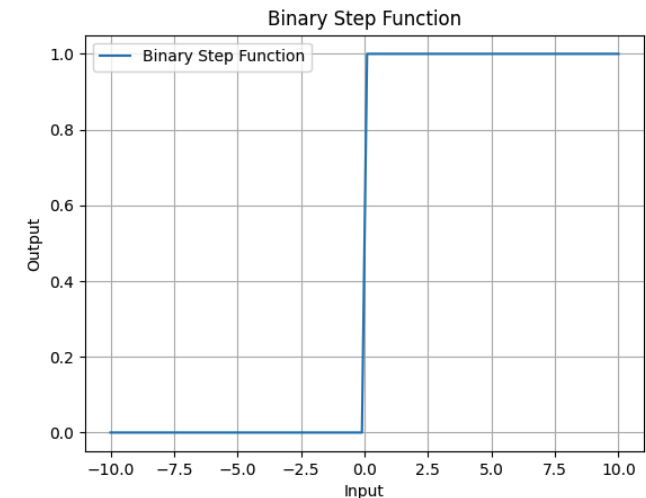
$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$

Advantages

- Very simple and computationally inexpensive
- Suitable for early perceptron models
- Clear binary decision boundary

Disadvantages

- Not differentiable at threshold
- Derivative is zero everywhere else
- Not suitable for deep learning



Common Activation functions

2. Bipolar Step Function

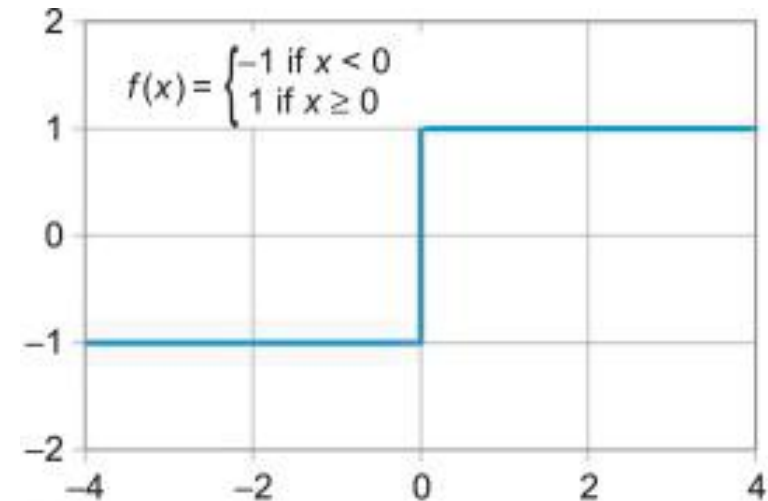
Advantages

- Very simple and computationally inexpensive
- Suitable for early perceptron models
- Clear binary decision boundary

Disadvantages

- Not differentiable at threshold
- Derivative is zero everywhere else
- Not suitable for deep learning

$$f(x) = \begin{cases} -1 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$



Common Activation functions

3. Sigmoid Function/ Logistic Sigmoid / Binary Sigmoid

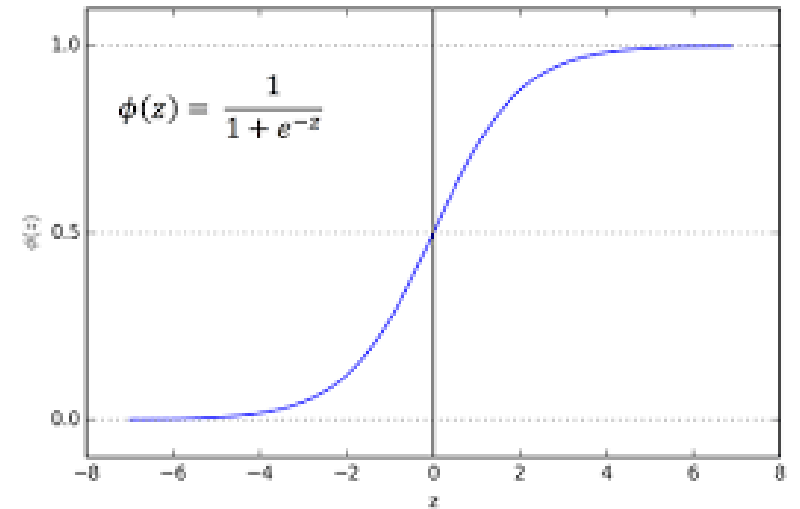
Advantages

- output range [0,1]
- Smooth and differentiable
- Interpretable as probability
- Non-Linear function
- Useful in binary classification (output layer)

Disadvantages

- Not zero-centered
- Slow convergence
- Saturating Function (vanishing Gradient problem)
- Computationally Expansive

$$f(x) = \frac{1}{1 + \exp(-\sigma x)}; f'(x) = \sigma f(x)[1 - f(x)]$$



Common Activation functions

5. Tanh Function

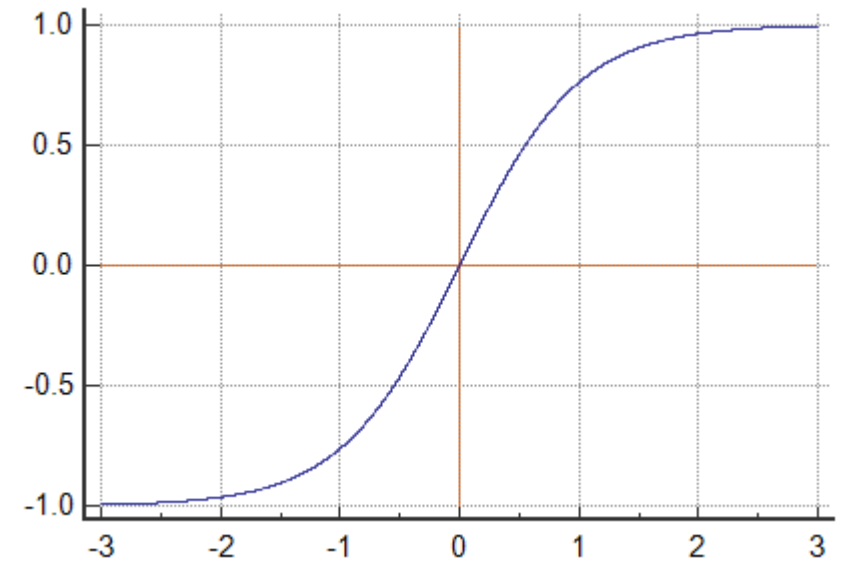
Advantages

- output range $[-1, 1]$
- Smooth and differentiable
- Non-Linear function
- Zero-centered

Disadvantages

- Slow convergence
- Saturating Function (vanishing Gradient problem)
- Computationally Expansive

$$f(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}; \quad \underline{f'(x) = [1 + h(x)][1 - h(x)]}$$



Common Activation functions

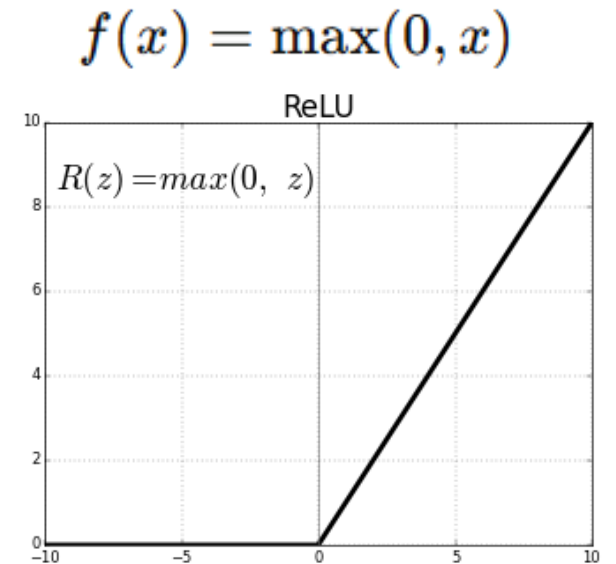
6. ReLU Function

Advantages

- Computationally simple
- Reduces vanishing gradient problem
- Fast training

Disadvantages

- "Dying ReLU" problem (neurons may stop activating)
- Not zero-centered



Common Activation functions

7. Softmax Function

The softmax activation function converts a set of numbers into a probability distribution, mainly used in multi-class classification.

For inputs $z_1, z_2, \dots, z_n$:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

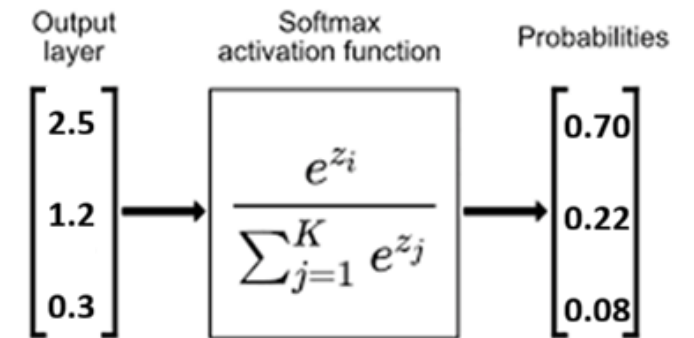
- A perceptron only handles binary classification
- What if we have 3 classes instead of 2? (Class A, Class B and Class C)
- A single perceptron cannot handle this directly.

Common Activation functions

7. Softmax Function

The softmax activation function converts a set of numbers into a probability distribution, mainly used in multi-class classification.

- Instead of one perceptron, use multiple perceptrons:
 - One for each class
 - Each gives a score
- Example outputs:
- $Z = [z_1, z_2, z_3] = [2.5, 1.2, 0.3]$
- These are just scores, not probabilities



- Class A → 70% chance
- Class B → 22%
- Class C → 8%

Common Activation functions

7. Softmax Function

Converts any real values into numbers between 0 and 1. Ensures all outputs sum to 1. Highlights the most likely class.

Advantages

- Probabilistic output
- Works well for multi-class problems
- Differentiable
- Emphasizes differences

Disadvantages

- Not zero-centered
- Sensitive to large values
- Overconfidence
- Classes are interdependent

Loss Function

- A loss function measures the discrepancy between the true value y and the predicted value.

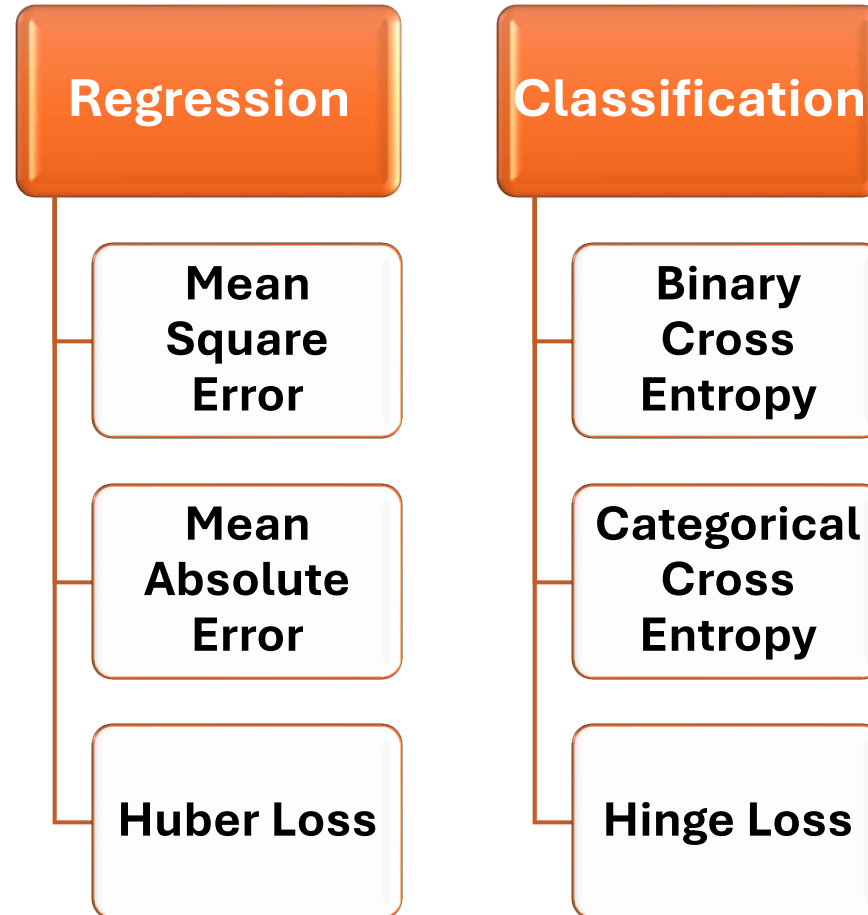
$$L = L(y, \hat{y})$$

- It quantifies how wrong the model's prediction is.
- **Importance**
 - Provides the objective to minimize during training.
 - Guides weight updates through backpropagation and gradient descent.
 - Directly influences model behavior and performance.
 - Aligns training with the desired prediction goal (e.g., accuracy, robustness).

Loss Function

- “*You cannot improve what you cannot measure*” –Peter Drucker
- The loss function acts as the “**eye**” of a machine learning **algorithm**: it observes how far the model’s predictions are from the true values and guides the **adjustment of weights** to improve performance.
 - Helps the model **learn the correct direction** during training.
 - Minimizing the loss ensures the model **fits the data accurately**.

Loss Function



Loss Function Vs. Cost Function

Loss Function	Cost Function
Measures error on a single training example	Aggregates loss over the entire dataset
Guides learning for one example	Guides learning for overall model optimization
$L(y, \hat{y})$	$J(\theta) = \frac{1}{m} \sum_{i=1}^m L(y^{(i)}, \hat{y}^{(i)})$
Squared error for one example	Mean Squared Error (MSE) over all examples

Loss Function

- Example using MSE
- **Loss Function** → Single training example
 - $Y=2$, $\hat{y} = 2.5$ $(y-\hat{y})^2 = 0.25$ now adjust weights
- **Cost Function** → Entire data

$$\text{MSE} = J(\theta) = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

$$\text{MSE} = \frac{0.25 + 0.04 + 0.04}{3} = 0.11$$

- Then adjust weights

x	y (true)	$\hat{y}$ (predicted)
1	2	2.5
2	3	2.8
3	4	4.2

Loss Function

1) Mean Square Error

- Measures the difference between the actual values and the predicted values

$$\text{MSE} = (y - \hat{y})^2$$

- **Advantages:**

- Penalizes large errors heavily → encourages accurate predictions.
- Easy to interpret.
- Differential
- Avoid Local optima

- **Disadvantages:**

- Sensitive to outliers.
- May exaggerate errors if target values vary widely.
- Error unit (squared)

Loss Function

2) Mean Absolute Error

- Mean Absolute Error measures the difference between the actual values and predicted values

$$MSE = |y - \hat{y}|$$

- **Advantages:**

- Less sensitive to outliers than MSE
- Easy to interpret
- Simple and intuitive
- same unit

- **Disadvantages:**

- Not differentiable at zero (though subgradient methods handle it)
- Does not penalize large errors as strongly as MSE
- May lead to slower convergence in gradient-based optimization

Loss Function

3) Huber Error

- It combines the advantages of MSE and MAE. It behaves:
 - Like MSE for small errors (no outlier)
 - Like MAE for large errors
- It reduces sensitivity to outliers while remaining smooth for optimization.

$$L_{\delta}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2, & |y - \hat{y}| \leq \delta \\ \delta (|y - \hat{y}| - \frac{1}{2}\delta), & |y - \hat{y}| > \delta \end{cases}$$

Where:

- δ = threshold parameter
- Small errors $\rightarrow$ quadratic penalty
- Large errors $\rightarrow$ linear penalty

Loss Function

4) Binary Cross-Entropy

- It is a loss function used for binary classification problems.
- It measures the difference between the true label $y \in \{0,1\}$ and the predicted probability $\hat{y} \in (0,1)$.

0

$$L = - [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

How It Works

- If $y = 1$:

$$L = -\log(\hat{y})$$

- If $y = 0$:

$$L = -\log(1 - \hat{y})$$

Wrong confident predictions are **penalized heavily**.

Loss Function

4) Binary Cross-Entropy

- It is a loss function used for binary classification problems.
- It measures the difference between the true label $y \in \{0,1\}$ and the predicted probability $\hat{y} \in (0,1)$.

$$L = - \overset{0}{\boxed{y \log(\hat{y})}} + (1 - y) \log(1 - \hat{y})$$

How It Works

- If $y = 1$:

$$L = -\log(\hat{y})$$

- If $y = 0$:

$$L = -\log(1 - \hat{y})$$

Wrong confident predictions are **penalized heavily**.

Loss Function

4) Binary Cross-Entropy

- It is a loss function used for binary classification problems.
- It measures the difference between the true label $y \in \{0,1\}$ and the predicted probability $\hat{y} \in (0,1)$.

$$L = - [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

- **Advantages:**

- Suitable for binary classification
- Works naturally with Sigmoid activation
- Strongly penalizes confident wrong predictions
- Differentiable and suitable for gradient descent

- **Disadvantages:**

- Can produce very large loss if prediction is extremely wrong
- Requires probability outputs (not raw logits unless modified)
- Multiple local optima

Loss Function

5) Categorical Cross-Entropy

- It is a loss function used for multi-class classification problems where each sample belongs to one of K classes.
- It measures the difference between the true class distribution and the predicted probability distribution.

$$L = - \sum_{j=1}^K y_j \log(\hat{y}_j) \quad , \text{ where K is number of classes}$$

- **How It Works**

- Uses Softmax activation in the output layer.
- Only the probability of the true class contributes to the loss.
- Heavily penalizes confident wrong predictions.

Loss Function

5) Categorical Cross-Entropy

- It is a loss function used for multi-class classification problems where each sample belongs to one of K classes.
- It measures the difference between the true class distribution and the predicted probability distribution.

$$L = - \sum_{j=1}^K y_j \log(\hat{y}_j) \quad , \text{ where } K \text{ is number of classes}$$

- **Advantages:**

- Standard loss for multi-class classification
- Works naturally with Softmax
- Differentiable and suitable for gradient-based optimization

- **Disadvantages:**

- Sensitive to extremely wrong confident predictions
- Requires one-hot encoded targets
- Can be affected by class imbalance